

TOPIC 3: PROGRAMMING ENVIRONMENT

CONCEPT:

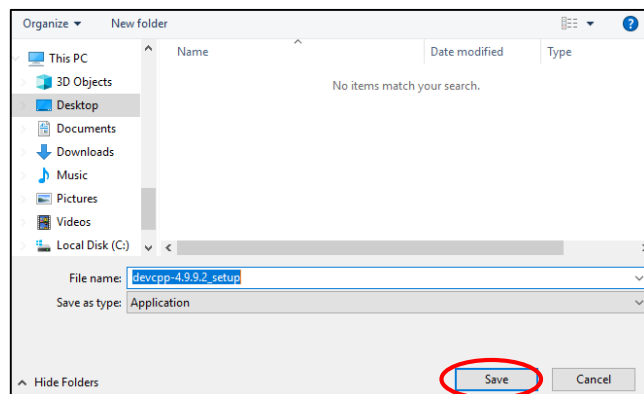
Part I: Download and Install Dev C++

(Do it yourself, install the software into your notebook)

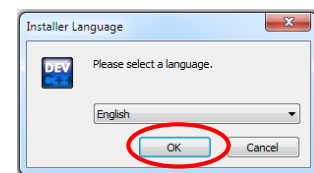
1. Click this link to download the installer
<https://sourceforge.net/projects/orwelldevcpp/>.
Once the site open, click **Download**.



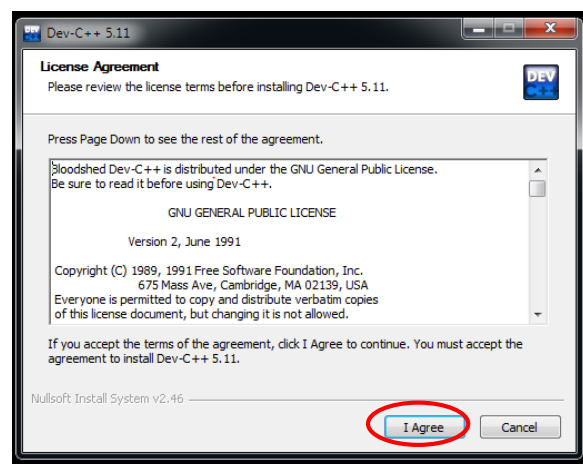
2. Save the file into any destination folder.



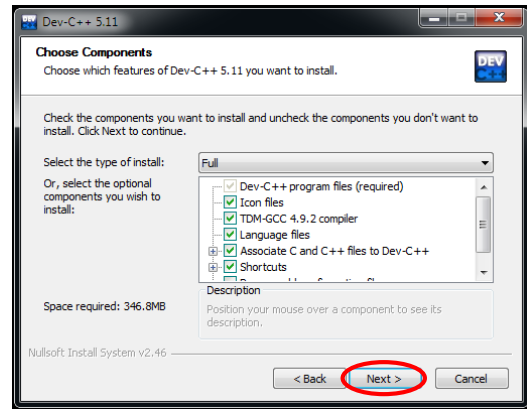
3. Select language, then click **OK**.



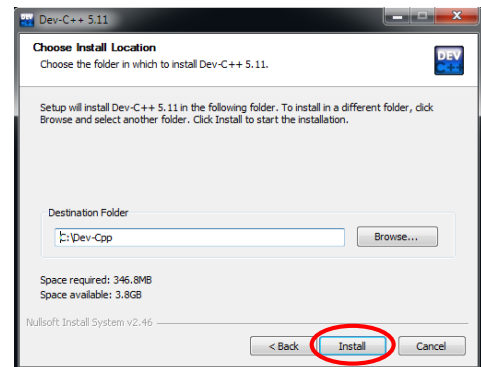
4. License Agreement: click **"I Agree"**:



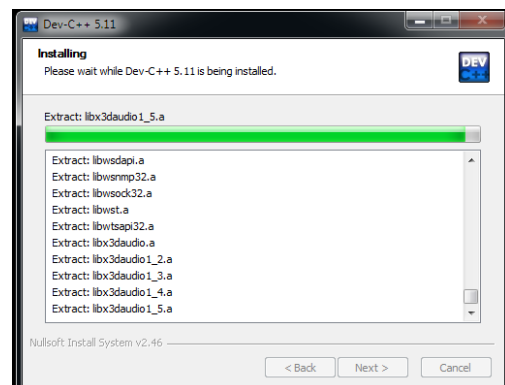
5. Choose Components, then click “**Next**”.



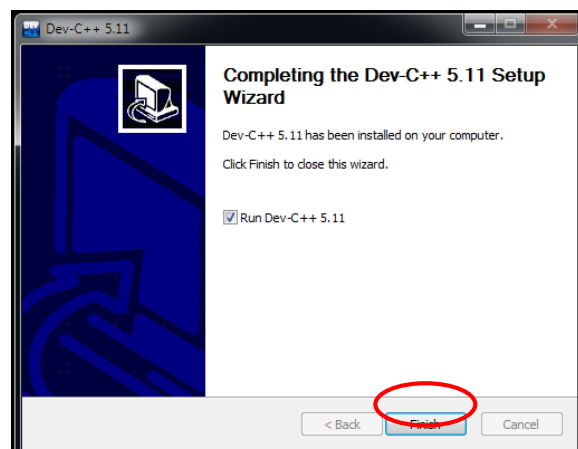
6. Choose Install Location: type location C:\Dev-Cpp, then Click “**Install**”.



7. Installation in progress, downloaded files will be extracted and the installation process started.

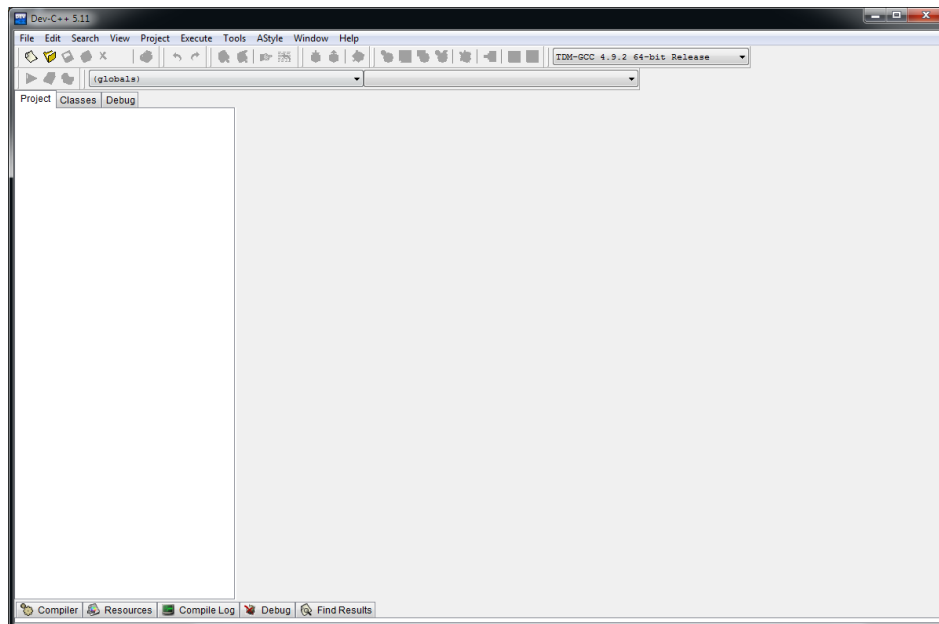


8. Click “**Finish**” to complete installation.

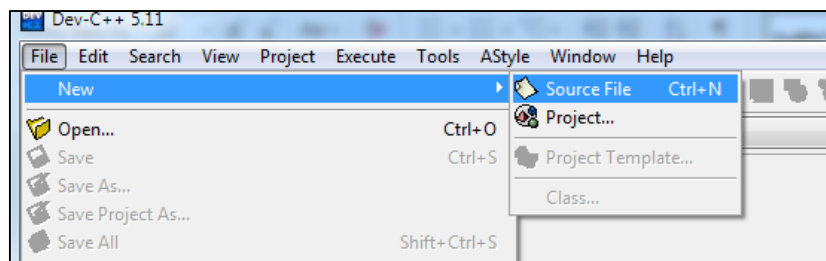


Part II: Edit a Program

1. Click Dev-C++ icon  to start the program.

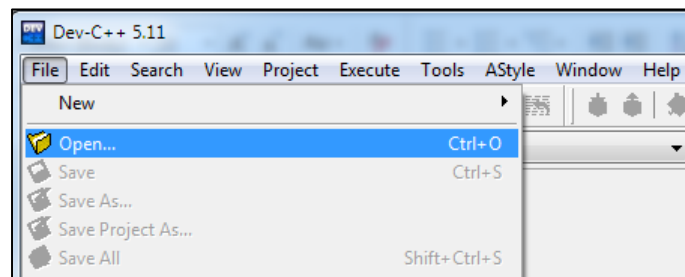


2. Click “**File**”, “**New**” and “**Source file**” to start writing a code.

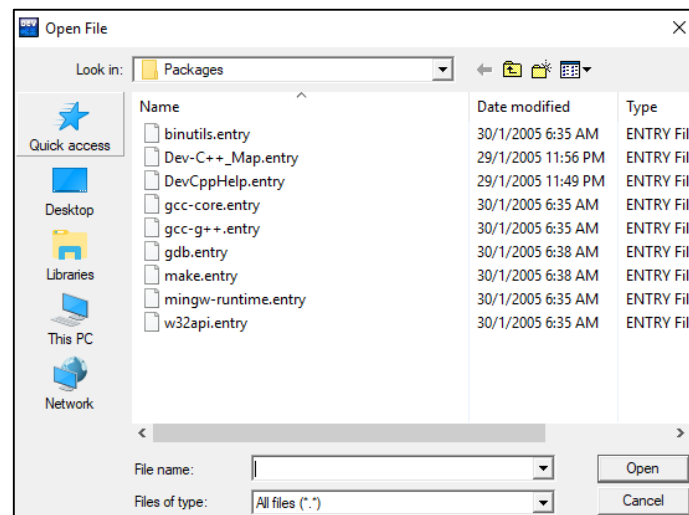


Note: User will be presented with blank space to write their code.

3. If user want to modify an existing code, do the following step:
 - a. Click **File**, then click **Open**.

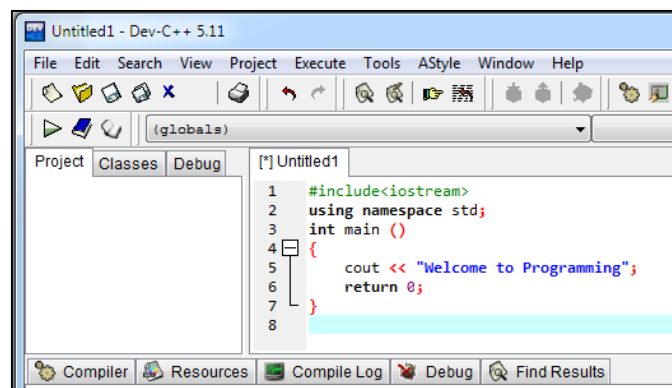


- b. Select the file needed to be open, click “**Open**”.



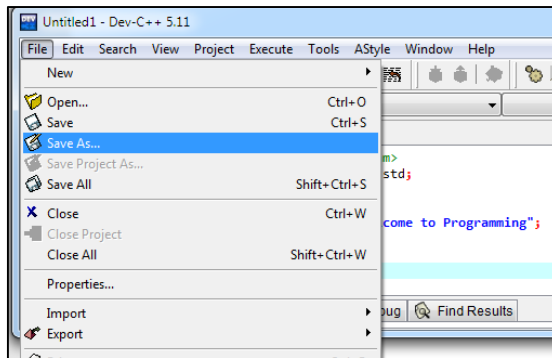
4. After Step 2, write the following code into the editor:

```
#include<iostream>
using namespace std;
int main ()
{
    cout << "Welcome to Programming";
    return 0;
}
```

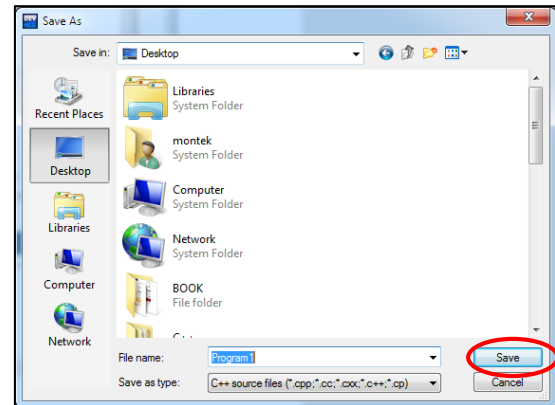


5. Save the program with filename “program1”.

a. Click “File” > “Save As ...”

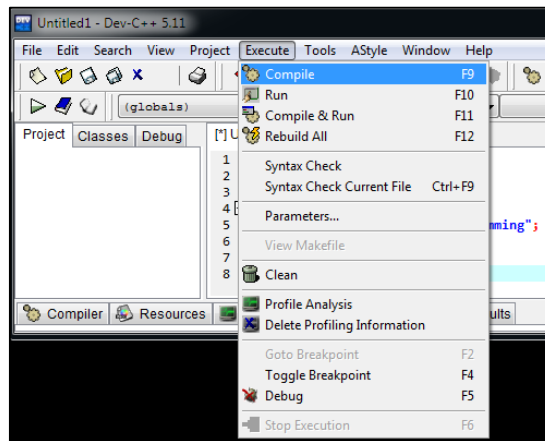


b. Select folder and give the program a name and click “Save”.

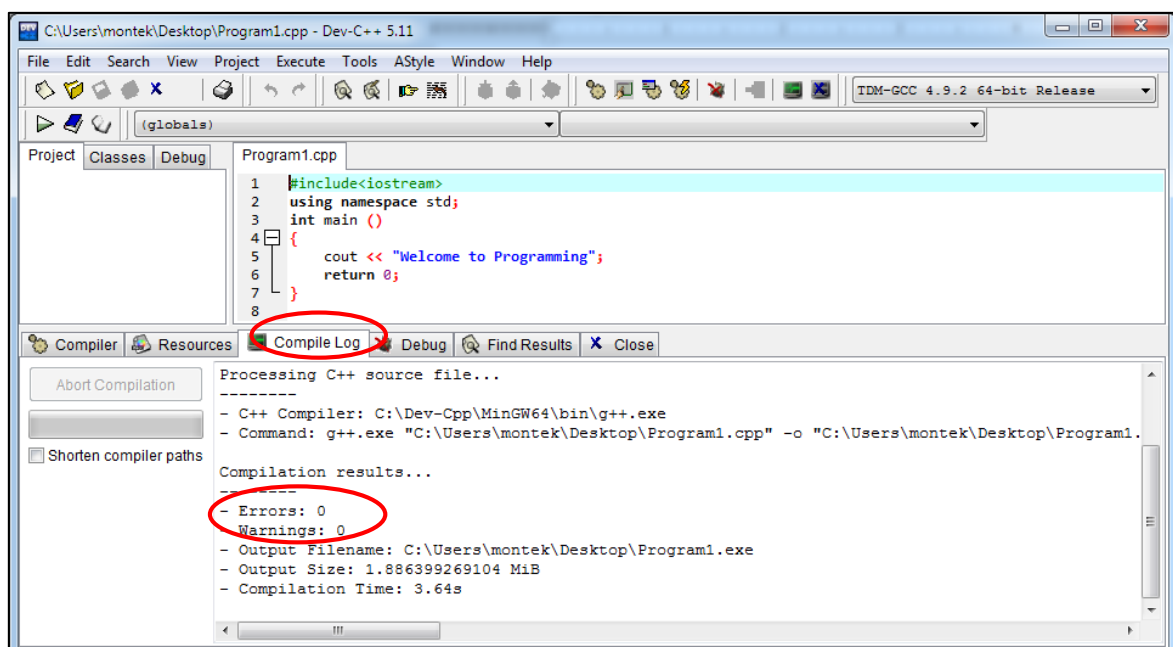


Part III: Compile the Program

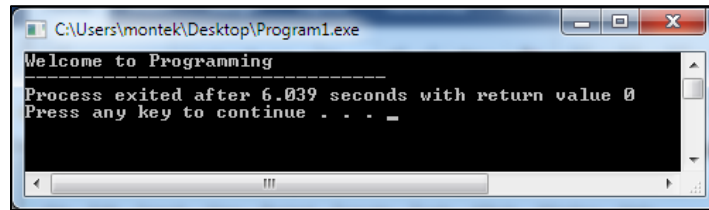
1. Click “Execute” > “Compile” or “Compile and Run”.



2. After the compilation, compilation status will be shown in “Compile Log” tab.

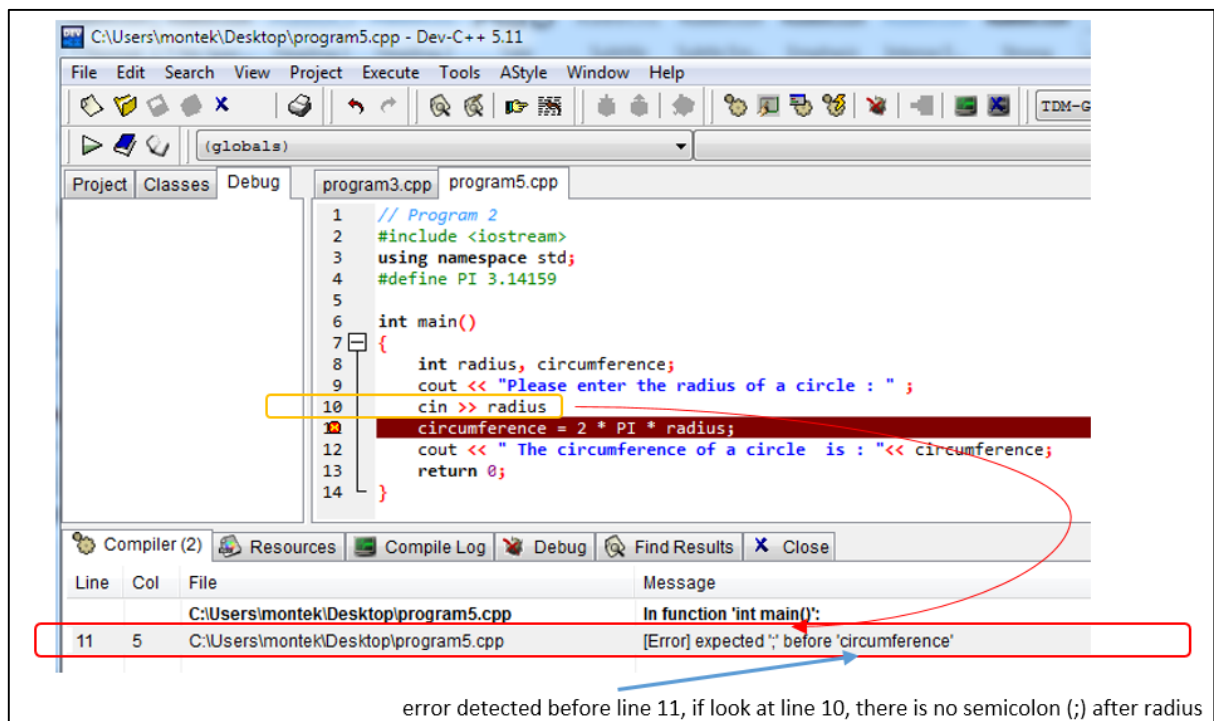


2. Two option during compilation, either “**Compile**” followed by “**Run**”, or perform the two operation with a single click of “**Compile and Run**”. When the program is running, it’s output will be displayed on a separate Windows.

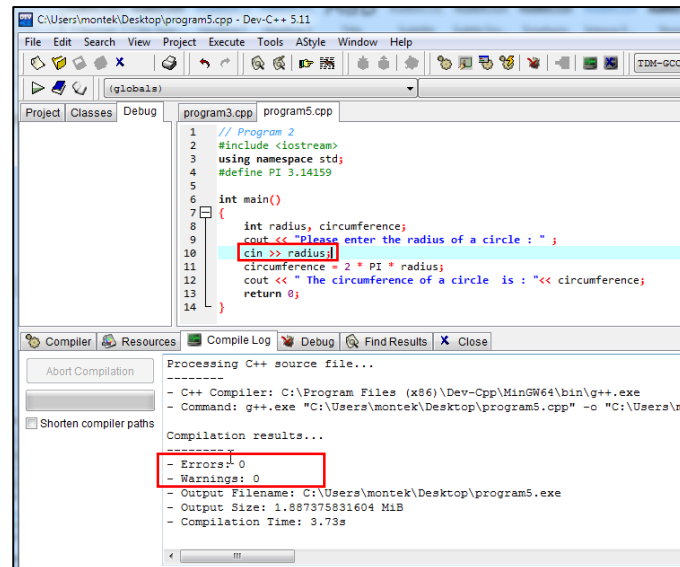


Part IV: Oops, Error...

1. While developing a program, there might be some errors. There are two types of error
 - a. Syntax Error
 - b. Logic error
2. Syntax error
 - a. Error in term of syntax/ formatting, **detected during compilation process**. Compiler will generate an error message. The error message will help the programmer to determine the line that causes the error and type of error.



- b. Programmer have to edit the source code, to remove the error. After adding ‘;’ at line 10, the source code is compiled again without an error.



3. Logic error

Logic error might not have detected by the compiler; the error will appear during execution of the program.

Write the following code into DEV-C++ editor.

```
// Program 2
#include <iostream>
using namespace std;
#define PI 3.14159

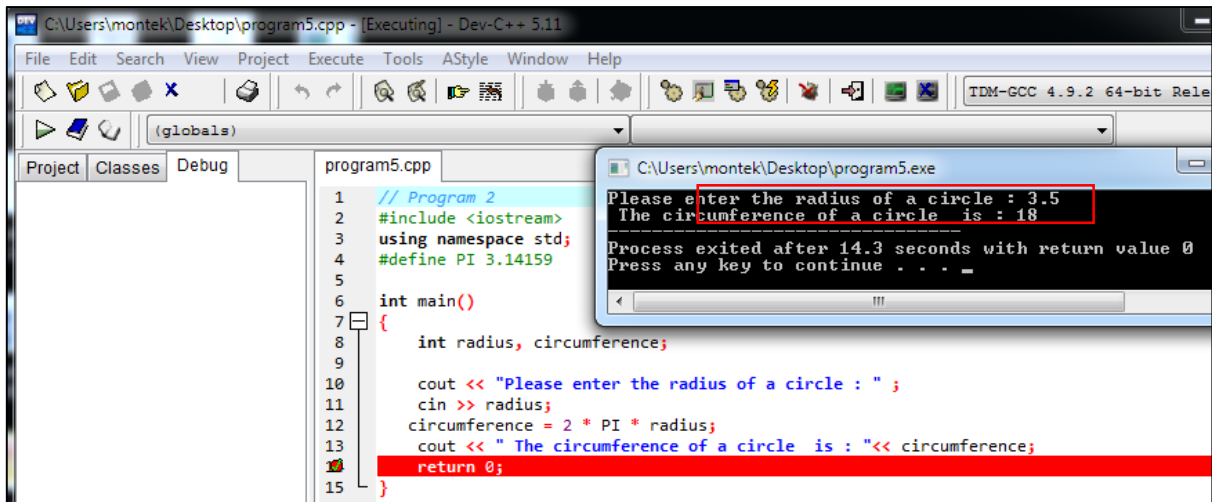
int main()
{
    int radius, circumference;

    cout << "Please enter the radius of a circle: " ;
    cin >> radius;
    circumference = 2 * PI * radius;
    cout << " The circumference of a circle is: " << circumference;
    return 0;
}
```

The program supposes to calculate the circumference of a circle for a radius entered by user.

Compile, then Run the program. When asked for radius, enter 2.58. Verify the answer produced by the program.

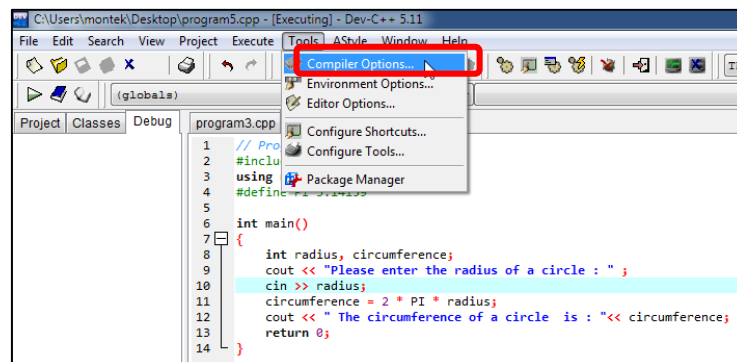
Diagram below show the output of the program when the radius is 3.5, the program calculate the circumference as 18, which is WRONG!



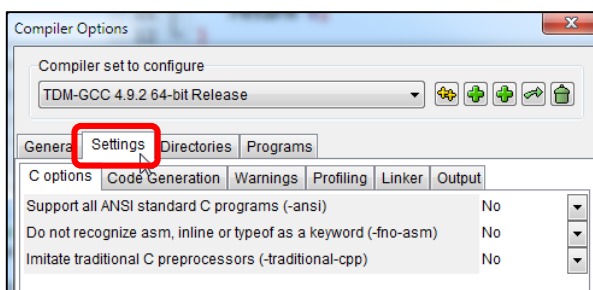
Part V: Configure Compiler to Support Debugging

To find out what causes the error, need the help of debugger. Before that, DEV-C++ must be configured properly so that it can debug a program. So configure the Compiler first.

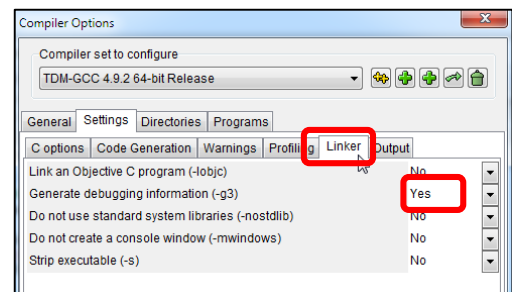
1. Go to the "Tools" menu and select "Compiler Options".



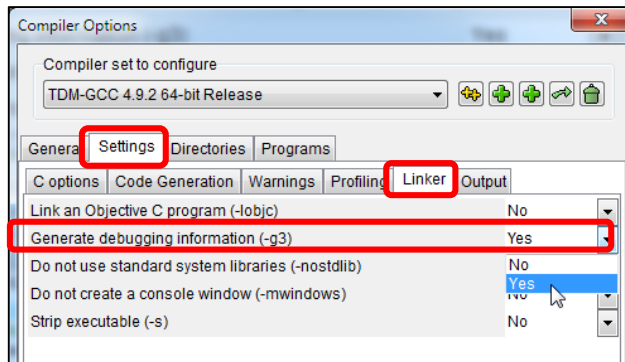
2. In the "Settings" tab...



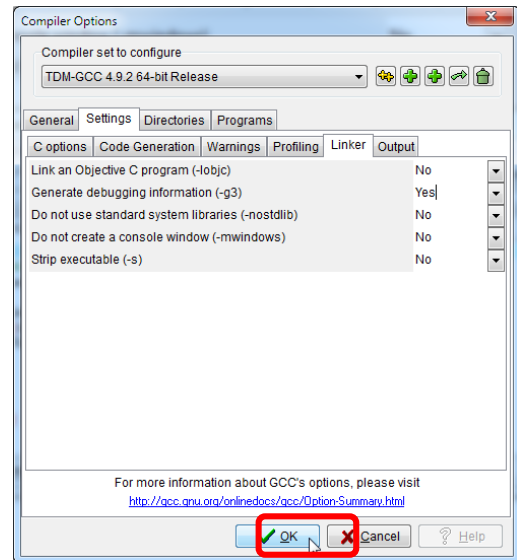
3. Click on the "Linker" Tab



4. Be certain that "**Generate debugging information**" is set to "Yes". If not, change it to Yes by clicking the triangle and click Yes.

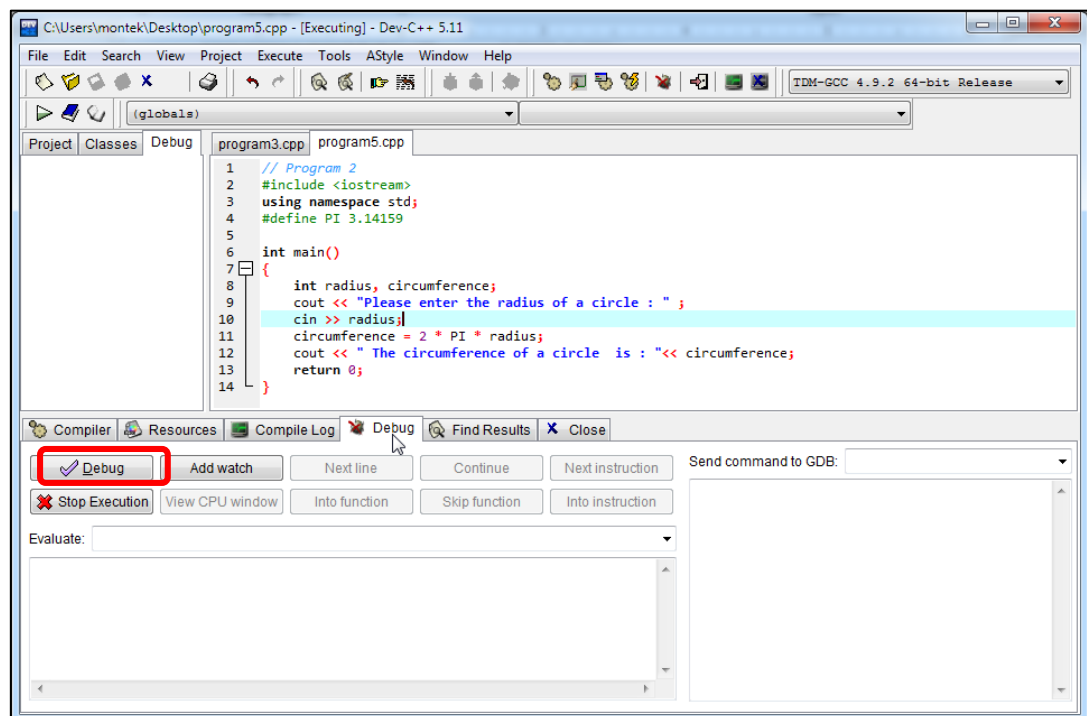


5. Click "OK".

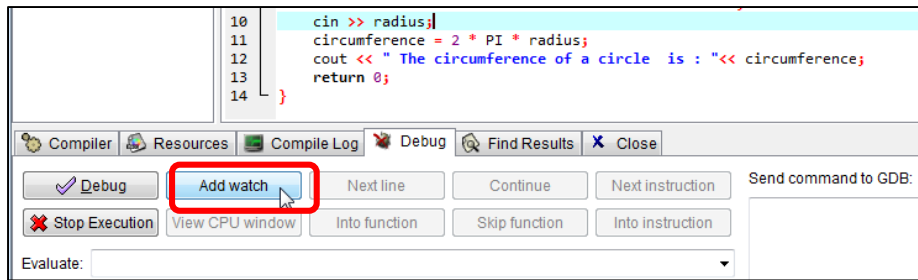


Part VI: Debugging

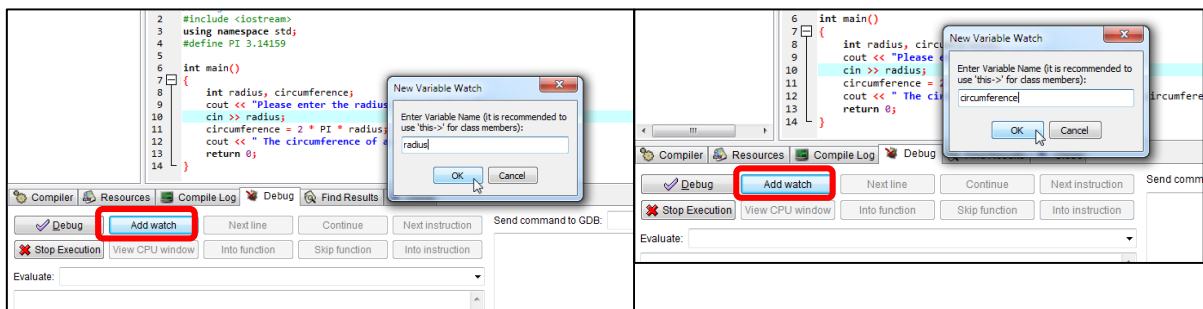
1. Please be certain that the debugger has been configured. Click **Debug**.



To debug need to monitor the value of variables, to determine whether the value is as expected. Click "**Add watch**" to monitor the value of variables.

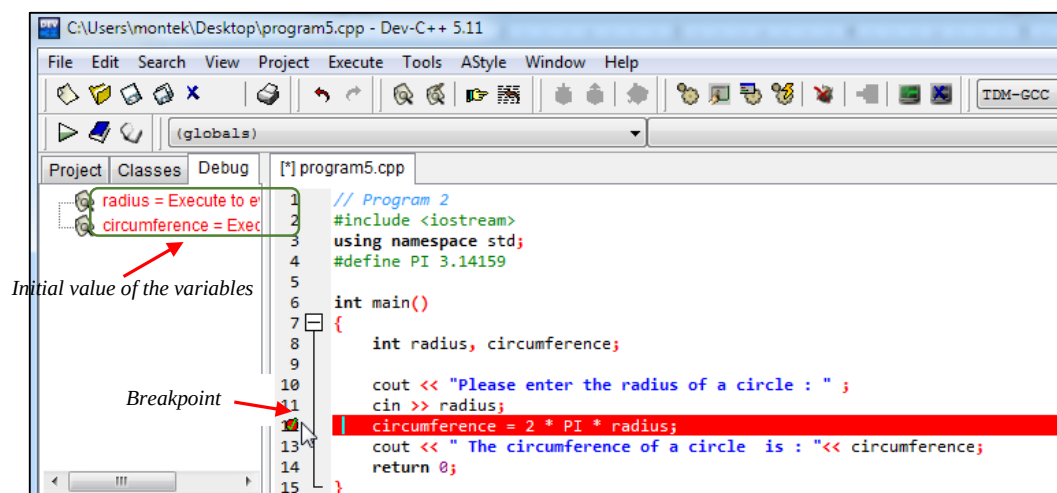


The value that need to be monitored are radius and circumference, so key in radius and circumference, after you click Add watch.

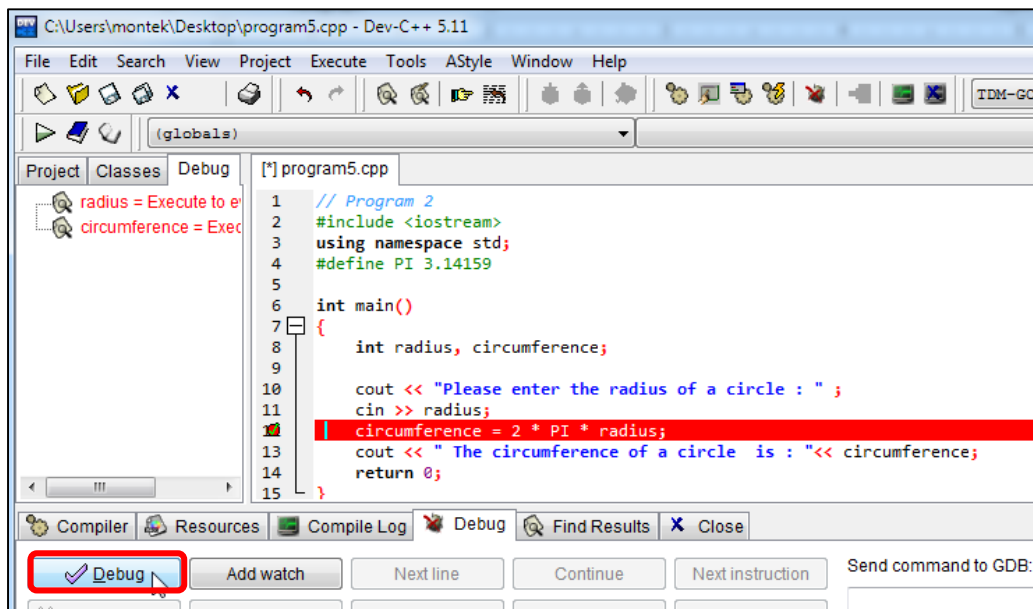


After **Add watch**, the variables appear at Debug windows, initially it's value is undefined. Execution of the program need to be stopped at certain point in the program so that the value of variables can be monitored.

In this example need to stop it at line 12. The program will be executed up to line 11 and stop at line 12. This will allow monitoring of the value of radius that is input by user. This can be achieved by setting the breakpoint at line 12, by clicking line number 12.

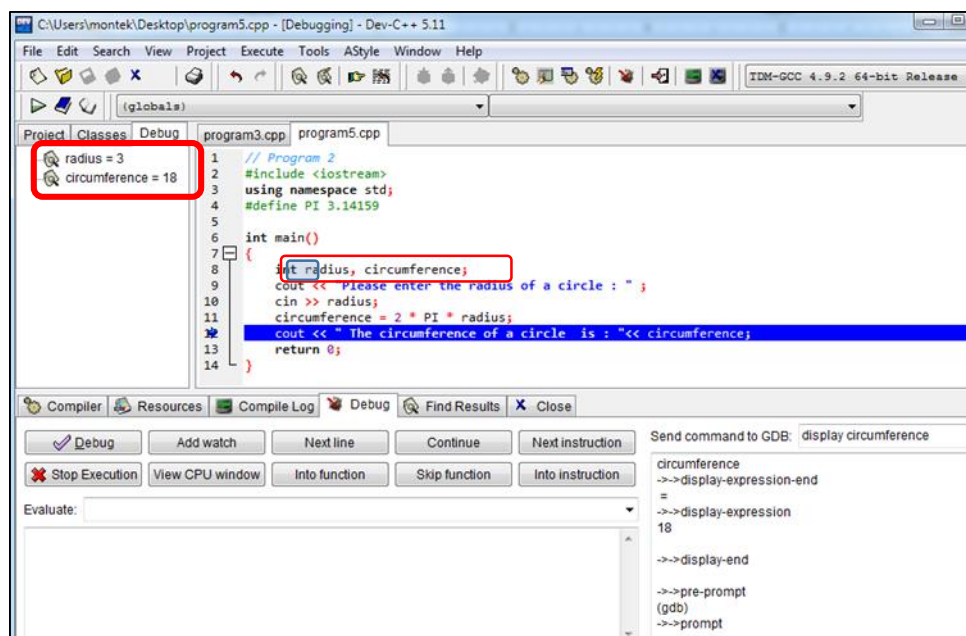


2. Click **Debug**



Clicking Debug will execute up to line 11. Click **Next line** will execute the program line by line. The blue line shows the code to be executed next, the red line is where the breakpoint is.

From the output it can see that a user entering the value of 3.5 as the radius of the circle, but if we look at the value of radius monitored by the debugger is 3. Why?

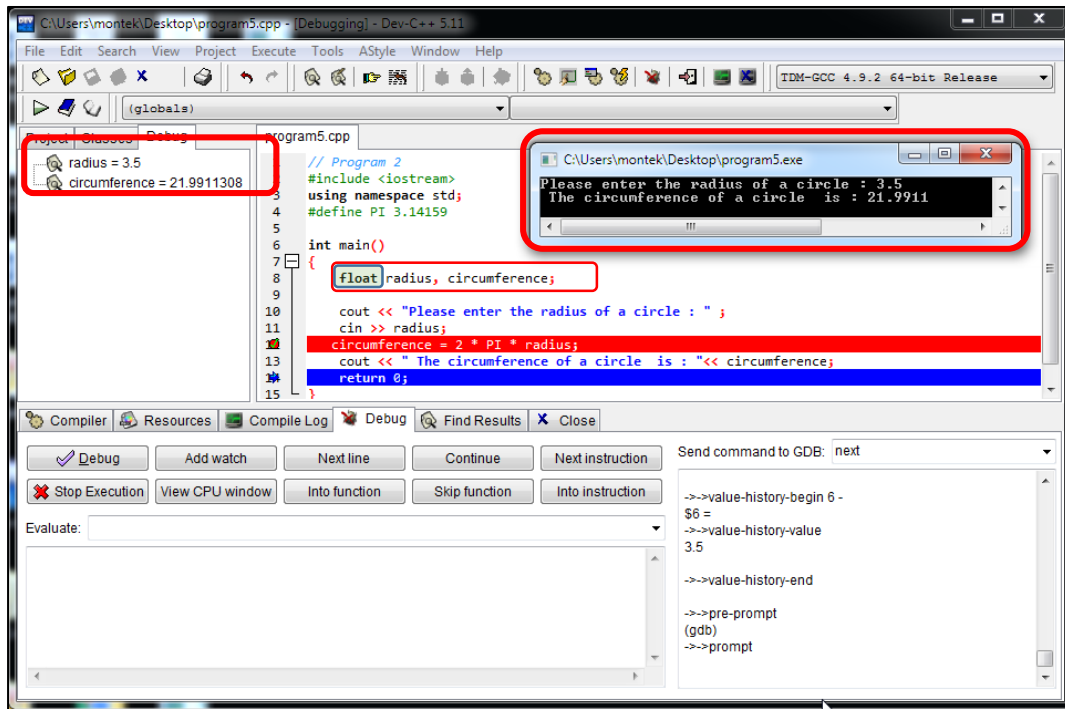


Variable radius is declared as an integer. Integer cannot hold a fractional number, user inputting 3.5 will be converted to 3 and stored in variable radius.

For circumference, the problem is the same, it is an integer variable that cannot hold fractional number.

By using a debugger, the problem can be tracked. Both variables cannot be integer, because its value can have a fractional part.

The solution is to change the variables to **float**, which can accept and process numbers with fractional parts. Once the changes made, recompile and run the program once again. This time the output is as expected.

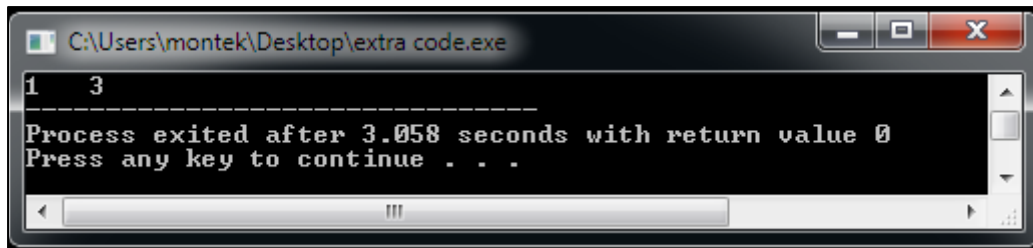


Part VII: Extra Activities

1. Key in the following code to DEV C++ editor, do not include the linenummer.

```
1  #include<iostream>
2  using namespace std;
3
4  int main()
5  {
6      int ode;
7
8      odd = 1;
9      while (odd != 4){
10         cout << odd << " "; // print the odd number
11         odd = odd + 2;      // add number with 2 to get next odd number
12     }
13     return 0;
14 }
15
```

2. The program supposes to print the value of odd numbers starting with 1 up to a number less than 4.



The expected output

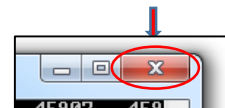
a. Compile the program. Did you find any syntax error?

- Syntax error #1 : _____
- Syntax error #2 : _____
- Syntax error #3 : _____

b. Once all syntax error (if any) has been remove, run the program.

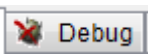
- Did the program stop? _____
- What type of error ? _____

if the program didn't stop, close the window by clicking

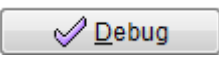


3. Debug the program to find out the problem.

a. Set breakpoint. Click at line number 8.

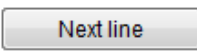
b. Click .

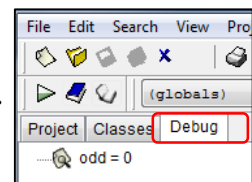
c. Click . Type **odd**. click OK.

d. Click . Your program will be executed and stop at line number 8.

Blue line indicates that this line will be executed next.

e. 1st iteration: Pay attention on variable **odd** value in debug windows.

- Click . What is the value of **odd**? _____
- Click Next line twice. The blue line will be at line number 12.
- Click Next line again. What is the new value of **odd** ? _____



f. 2nd iteration: Pay attention on variable **odd** value in debug windows.

- Click Next line twice. The blue line will be at linenumber 12.
- Click Next line again. What is the new value of **odd** ? _____

g. Is the value of **odd** at (f-ii) greater than 4 ? _____

h. We want to print **odd** number less than 4. Will the expression **odd != 4**; which is odd not equal to 4 will ever be true? **Y / N**

- i. Now I hope you can see the cause of the problem. To solve the problem, try to replace the expression `!=` with one of the following:

<code>=</code> (equal) <code>>=</code> (greater or equal), <code><=</code> (less or equal) <code><</code> (less than) <code>></code> (greater than)
--

- j. Compile the modified program and run it again.
k. Write down the output.

**I hear and I forget.
I see and I remember.
I do and I understand.**